School of
Computing Science

CSC1107 – Operating Systems

Assessed Coursework

Course Name	**CSC1107 – Operating Systems**			
Coursework No.	**Assignment**			
Deadline	**Time**	**23:59**	**Date**	**10 July 2026**
% Contribution to final course mark			**25%**	
Solo or Group	**Solo**		**Group**	**√**
Submission Instructions	**Submit your assignment report to the Dropbox of LMS xSiTe**			
Please Note: This Coursework cannot be Re-Done				

Code of Assessment Rules for Coursework Submission

Please work the assignment by yourselves without copying from others. If any plagiarism is found, the works submitted will be awarded Grade F.

Deadlines for the submission of coursework which is to be formally assessed will be published in course documentation, and work which is submitted later than the deadline will be subject to penalty as set out below.

The primary grade and secondary band awarded for coursework which is submitted after the published deadline will be calculated as follows:

(i) in respect of work submitted not more than five working days after the deadline
   a. the work will be assessed in the usual way;
   b. the primary grade and secondary band so determined will then be reduced by two secondary bands for each working day (or part of a working day) the work was submitted late.

(ii) work submitted more than five working days after the deadline will be awarded Grade F.

Penalties for late submission of coursework will not be imposed if good cause is established for the late submission. You should submit documents supporting good cause to Admin-In-Charge

1) Objectives

The objectives of the group project assignment are to reinforce and practice the knowledge learnt in CSC1107 – Operating Systems, particularly on the Linux loadable kernel module (LKM), user space programming, peripherals, OS algorithms, and Bash shell script development on Raspberry Pi. The Bash shell script can automate the execution of a series of Linux commands/tasks one by one. It can greatly increase operational efficiency with less human intervention.

We will also encourage you to explore how generative Artificial Intelligence (GenAI) could help improve your programming skills and approaches in the project assignments. Your analytics and reflections on the comparisons of your self-developed programming codes and GenAI generated codes will be needed.

Question

Peripheral or internal devices allow users to communicate with the computer. Examples of devices are: keyboards, monitors, hard disks, USB devices, printers, networks, etc. A driver is needed to interface with the OS that manages communication with peripheral devices; thus, they are usually called device drivers, shown in Fig. 1.

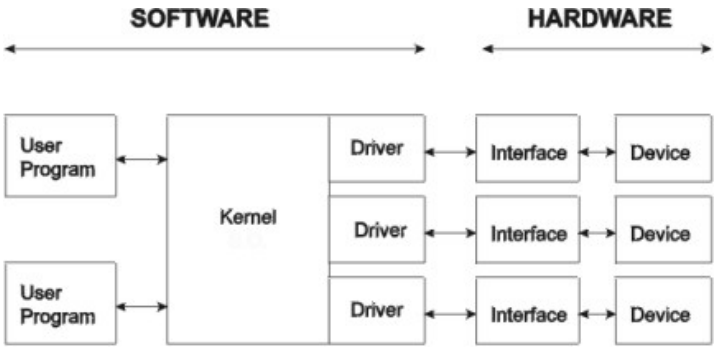

Figure 1. Simple locations for Kernel and device drivers

In this question, we will develop Linux loadable kernel modules, user space C programs and Bash shell script in your Raspberry Pi. We have learnt the loadable kernel modules of Linux OS in the tutorials of Week 1 and Week 2. Please refer to the online tutorial documents for your further references:

(a). Writing device drivers in Linux: A brief tutorial
(more details on the web link here) in [1]

(b). https://www.oreilly.com/openbook/linuxdrive3/book/ch03.pdf

Note: Check Reference section

Please develop a loadable kernel module and a user space C program to communicate between the kernel and user space, which involves a peripheral device of the Raspberry Pi with the *read()* system call (for one or multiple read operations) and the *write()* system call (for one or multiple write operations).

For your consideration, you are encouraged to use open sources, GenAI approach, or relevant methods published at Internet. You do not have to develop from scratch by yourselves. Please perform searches over the internet for seek for similar solutions followed by revisions by yourselves.

- Example 1 for your consideration.

For example, when a USB device (e.g., a USB storage device, mouse, keyboard, or camera) is plugged into the Raspberry Pi's USB port, your bash shell script will load a loadable kernel module, and launch the user space program to send a *write()* system call, to send a text message "Hello World from the user space" to the loadable kernel module. Next, it will send a *read()* system call, to get a message "Hello World from the kernel space". Print these text messages to screen and kernel space, such that your bash shell script can display these messages using the *dmesg* command. Please use Google search to look for methods at Internet, e.g., Google search key workds "*linux loadable kernel module when a usb device is connected raspberry pi*".

- Example 2 for your consideration.

As another example, when an Ethernet cable is plugged into Raspberry Pi's RJ45 port, your bash shell script will load a loadable kernel module, and launch the user space program to send a *write()* system call, to send a text message "Hello World from the user space" to the loadable kernel module. Next, it will send a *read()* system call, to get a message "Hello World from the kernel space". Print these text messages to screen and kernel space, such that your bash shell script can display these messages using the *dmesg* command. Please use Google search to look for methods at Internet, e.g., Google search key workds "*linux loadable kernel module when an ethernet cable is connected raspberry pi*".

These two above examples are for your reference. While you could also consider other examples, involving an external peripheral device, such as WiFi, Bluetooth, HDMI, LCD screen, etc.

Please use the *makefile* utility to compile the derive drive, and insert the loadable kernel module in the Linux kernel. Please use the *gcc* command to compile the user

space C program. Please automate these tasks using a Bash shell script, such that users only need to launch the Bash shell to get these tasks completed one by one.

You may choose to start from scratch in programming, or you can google search for available similar solutions in C codes. Please cite the references in the report that you have used and refactor those codes. You are encouraged to use GenAI, e.g., ChatGPT or other platforms for helps to achieve the goal. But give us your detailed reflections and approaches how the GenAI helps you in this question. In your programming codes, please add clear and detailed comments, to help readers better understand your codes.

2) Assessment criteria

Assignments will be assessed according to the criteria for the Group Assessment and the Individual Assessment. Note that the criteria are subject to change depending on the progress throughout the trimester. Following timely submission, the submissions will be given a numerical mark between 0 (no submission) and 100 (perfect in every way) as the group assessment results. The numerical marks will then be converted to a grade.

Your marks for this assignment will be computed as follows:

Each group member grade = Group assessment * (weighted peer review score by each group member and his/her presentation performance in presentation video)

*Table 1. Group assessment**

*Group assessment will be weighted by peer review

Criteria	Weight
Solutions that produce the correct results.	25
Professional writing of the report with a clear logic and structure of your solutions. Quality of design documentation to describe your design in the step-by-step manner, to help readers better understand your design and how to achieve the same design following your documentations. (The design documentation is integrated into the report, as a part of the report.)	50
PowerPoint presentation and demo videos: • A clear and smooth demonstration and presentation of your solutions in about 8-12 minutes presentation video. • Every group member must give the speech on his/her respective portion in the assignment. • Highlight the strength and limitation of your solutions. • List down all group member's individual workloads and contributions in the report and presentation. • Reflection from each group member regarding their learning journey throughout the assignment.	25

Table 2. Individual contribution assessment

Each member needs to write self-reflections in the learning of CSC1107 – OS and learning in the assignment project, particularly on how GenAI helps you and your views on GenAI in learning programming tasks/projects in the submitted report. Each individual team member is required to implement a portion of the project design, documentation and report. Every group member must give the speech on his/her respective portion in the assignment. Therefore, a clear task distribution is required. Peer evaluation score will be used as part of the individual contribution assessment factor. '0' for the peer evaluation score means no contribution by this member at all. '100' for the peer evaluation score means perfect contribution by this member according to the mutually agreed work scopes. Comments are also required to be provided in each peer evaluation, to avoid some scenarios to mark down the contributing group members without the valid reasons.

3) Timeline and documents for submissions:

Final submission: **Friday, 10 July 2026, 23:59 pm** (7.5 weeks from now).

Each group is to submit these four separate files with file names embedded ***Px_Groupxx*** to your group's LMS xSITE Dropbox. Px means your lab sessions P1, P2 and P3. Groupxx means your group number in the lab session (Group1, Group2, etc.). Please only submit your final version of files to Dropbox.

1) the **WORD format final report** named as Px_Groupxx-report-CSC1107.docx, (do not copy and paste entire source codes in the report),
2) zip all the source codes (`.c, .h, .py, .sh` files) into a source code file, named as Px_Groupxx-codes.zip.
3) a group power point presentation video named as Px_Groupxx-CSC1107.mp4, with about 8-12 minutes length where all group members must give their speech on their respective portion of works in this group assignment.
4) Short demo videos named as Px_Groupxx-demo.mp4.

4) References:

[1] X. Calbet, “Writing device drivers in Linux: A brief tutorial,” *Free Software Magazine*, [online]. Available: https://faculty.winthrop.edu/domanm/csci411/Handouts/linux_device_driver_tutorial.pdf, [accessed 15 May 2025].

[2] J. Corbet, A. Rubini, G. Kroah-Hartman, “Linux Device Drivers,” Third Edition, Publisher: O’Reilly Media, Inc., 2019. [online]. Available: https://www.oreilly.com/openbook/linuxdrive3/book/ch03.pdf, [accessed 15 May 2025].

5) Appendix: Project Topics

These project ideas are designed with reference to the introductory Linux kernel module and character device driver practical labs from CSC1107 Operating Systems. The introductory labs cover Linux kernel building, loadable kernel modules, *Makefiles*, *printk(), dmesg*, module insertion/removal, and character device communication between kernel-space and user-space applications.

The projects below extend the same technical foundations into practical Raspberry Pi 4 industry-relevant applications using:

- Raspberry Pi OS (Raspbian 64-bit)
- Linux kernel modules (*.ko*)
- Character or block device drivers
- User-space applications in C
- USB device interaction
- GPIO and Sense HAT integration
- Linux logging using *printk(), dmesg, journalctl*
- *Makefile* and kernel module compilation workflow

Each project also includes one advanced challenge component focused on networking, cybersecurity, or secure systems integration.

Common Technical Expectations for All Projects

Minimum Core Technical Requirements

Each project should demonstrate:

1. Linux kernel module development
2. Character or block device driver creation
3. Communication between user-space and kernel-space
4. Device node creation under */dev*
5. Proper use of *printk()* and *dmesg* logging
6. *Makefile*-based module compilation
7. Safe loading/unloading using *insmod* and *rmmod*
8. Error handling and debugging
9. Demonstration on Raspberry Pi 4 running Raspberry Pi OS
10. Technical documentation and testing procedure

Common Demonstration Deliverables

Students should demonstrate:

- Successful module compilation
- Successful insertion/removal of module
- Proper detection of hardware device
- Functional communication between application and driver
- Logging visibility in *dmesg*
- User application interaction with */dev* device file
- Clean shutdown and resource release
- Technical explanation of kernel-space vs user-space operations
- Video presentation of your solutions in about 8-12 minutes

Project 1: USB Thumb drive Access Monitor Driver

Goal

Develop a character device driver that monitors insertion and removal of USB thumb drives and logs activity into the Linux kernel log.

Industry Relevance

- Endpoint monitoring
- Device access auditing
- Cybersecurity logging
- IT infrastructure management

Technical Focus

- USB subsystem interaction
- Character device driver
- Kernel event logging
- User-space status utility

Technical Challenges

- Detecting USB device insertion/removal
- Tracking vendor and product IDs
- Handling concurrent device events
- Synchronizing kernel logs with user-space application

Resources Needed

- Raspberry Pi 4
- USB thumb drive
- Raspberry Pi OS
- GCC and kernel headers

Video Demonstration

- Driver creates */dev/usbmon*
- Kernel logs display USB insertion/removal events
- User application displays connected USB device information
- *dmesg* output verification

Challenge Goal

Transmit USB insertion alerts to a remote syslog or MQTT server for centralized monitoring.

Project 2: USB Keyboard Activity Logger Driver

Goal

Create a character device driver that captures keyboard events and counts keystroke activity.

Industry Relevance

- Human-machine interface systems
- Industrial console monitoring
- Security event tracking

Technical Focus

- USB HID interaction
- Character device buffering
- Kernel interrupt handling concepts

Technical Challenges

- Reading keyboard input safely
- Avoiding kernel instability
- Implementing buffered event storage
- Synchronization between kernel and user-space

Resources Needed

- USB keyboard
- Raspberry Pi 4
- Linux kernel headers

Demonstration Requirements

- Driver records keyboard activity count
- User application retrieves statistics from `/dev/kbmonitor`
- `printk()` displays event activity
- Proper module insertion/removal

Challenge Goal (Advanced)

Add remote encrypted logging of keyboard statistics using TLS sockets.

Project 3: USB Mouse Movement Tracking Driver

Goal

Develop a character device driver that tracks mouse movement and button click activity.

Industry Relevance

- Operator activity monitoring
- Smart kiosk systems
- Human interaction analytics

Technical Focus

- USB mouse event parsing
- Character device communication
- Kernel-space buffering

Technical Challenges

- Parsing HID reports
- Managing real-time event updates
- Handling fast input streams

Resources Needed

- USB mouse
- Raspberry Pi 4
- Raspbian OS

Demonstration Requirements

- User application displays cursor movement statistics
- Driver logs click activity
- *dmesg* verification of events

Challenge Goal (Advanced)

Implement network streaming of movement statistics to a dashboard server.

Project 4: GPIO LED Controller Driver

Goal

Build a GPIO character device driver to control LEDs connected to Raspberry Pi GPIO pins.

Industry Relevance

- Industrial indicator systems
- Embedded control systems
- Smart automation devices

Technical Focus

- GPIO programming
- Character device interface
- *`ioctl()`* implementation

Technical Challenges

- Safe GPIO memory access
- Kernel-level timing control
- User-space command parsing

Resources Needed

- LEDs
- Resistors
- Breadboard
- Raspberry Pi 4

Demonstration Requirements

- LED control via */dev/gpioled*
- User application changes LED state
- Driver logs GPIO events

Challenge Goal (Advanced)

Add secure TCP remote LED control with authentication.

Project 5: Sense HAT Environmental Monitor Driver

Goal

Create a character device driver that exposes Sense HAT sensor readings to user-space applications.

Industry Relevance

- Environmental monitoring
- Industrial IoT
- Smart building systems

Technical Focus

- Sensor data handling
- Character device driver
- Kernel-user communication

Technical Challenges

- Polling sensor values
- Managing multiple sensor streams
- Buffer synchronization

Resources Needed

- Raspberry Pi Sense HAT
- Raspberry Pi 4

Demonstration Requirements

- Temperature and humidity retrieval through `/dev/senseenv`
- Real-time sensor display application
- Logging through `printk()`

Challenge Goal (Advanced)

Publish sensor readings securely to MQTT broker with authentication.

Project 6: Webcam Snapshot Capture Driver

Goal

Develop a driver-assisted image snapshot trigger system using a USB webcam.

Industry Relevance

- Smart surveillance
- Factory monitoring
- Edge AI systems

Technical Focus

- Video device interaction
- Character device triggering
- File operations

Technical Challenges

- Synchronizing image capture requests
- Managing memory buffers
- Handling USB video device communication

Resources Needed

- USB webcam
- Raspberry Pi 4

Demonstration Requirements

- Snapshot trigger through `/dev/camtrigger`
- User application saves captured image
- Logging and timestamp tracking

Challenge Goal (Advanced)

Implement encrypted remote image upload to server.

Project 7: SD Card Health Monitoring Block Driver

Goal

Create a block device monitoring driver for SD card access and performance statistics.

Industry Relevance

- Storage reliability monitoring
- Embedded systems maintenance
- Data center storage analytics

Technical Focus

- Block device interaction
- Read/write monitoring
- Kernel statistics reporting

Technical Challenges

- Intercepting storage operations safely
- Monitoring I/O activity
- Managing kernel memory usage

Resources Needed

- USB SD card adapter
- SD card
- Raspberry Pi 4

Demonstration Requirements

- Display read/write counts
- User application retrieves block statistics
- Kernel log monitoring

Challenge Goal (Advanced)

Add anomaly detection for suspicious storage activity.

Project 8: USB Device Whitelist Security Driver

Goal

Develop a security-oriented character driver that only permits approved USB devices.

Industry Relevance

- Industrial cybersecurity
- Endpoint protection
- Secure embedded systems

Technical Focus

- USB vendor/product ID filtering
- Device authorization logic
- Kernel access control concepts

Technical Challenges

- Managing approved device database
- Preventing unauthorized access
- Synchronization with user-space policy manager

Resources Needed

- Multiple USB devices
- Raspberry Pi 4

Demonstration Requirements

- Approved devices allowed
- Unauthorized devices blocked and logged
- User utility updates whitelist

Challenge Goal (Advanced)

Add SHA256 signed whitelist policy verification.

Project 9: GPIO Motion Detection Driver

Goal

Implement a GPIO-based motion detection driver using a PIR sensor.

Industry Relevance

- Smart security systems
- Building automation
- Occupancy detection

Technical Focus

- GPIO interrupts
- Character device notifications
- Event-driven kernel programming

Technical Challenges

- Interrupt debouncing
- Event queue implementation
- Preventing false triggers

Resources Needed

- PIR motion sensor
- Raspberry Pi 4

Demonstration Requirements

- Motion events logged in kernel
- User application receives alerts
- GPIO interrupt demonstration

Challenge Goal (Advanced)

Send motion alerts over network with secure authentication.

Project 10: GPIO Smart Fan Controller Driver

Goal

Create a GPIO-based cooling fan control driver using temperature thresholds.

Industry Relevance

- Embedded thermal management
- Industrial equipment cooling
- Edge computing systems

Technical Focus

- GPIO output control
- Sensor polling
- Character device interface

Technical Challenges

- Threshold management
- Timing synchronization
- Fan state transitions

Resources Needed

- GPIO fan module
- Temperature sensor or Sense HAT
- Raspberry Pi 4

Demonstration Requirements

- Fan auto-control based on temperature
- User-space configuration utility
- Kernel log output verification

Challenge Goal (Advanced)

Create remote web dashboard for thermal monitoring.

Project 11: USB Barcode Scanner Driver Interface

Goal

Build a character driver that interfaces with USB barcode scanners.

Industry Relevance

- Warehouse automation
- Logistics systems
- Retail systems

Technical Focus

- USB HID processing
- Device buffering
- Event logging

Technical Challenges

- Parsing scanner data streams
- Handling continuous scans
- Buffer overflow prevention

Resources Needed

- USB barcode scanner
- Raspberry Pi 4

Demonstration Requirements

- Barcode data accessible through `/dev/barcodescan`
- Logging of scanned data
- User application displays scan history

Challenge Goal (Advanced)

Transmit scans securely to inventory server.

Project 12: Secure Character Device Login Driver

Goal

Develop a secure character device driver requiring authentication before access.

Industry Relevance

- Secure embedded systems
- Authentication devices
- Access-controlled hardware

Technical Focus

- *ioctl()* authentication commands
- Kernel credential handling
- Device access control

Technical Challenges

- Password validation
- Preventing unauthorized reads/writes
- Managing session states

Resources Needed

- Raspberry Pi 4
- Linux kernel headers

Demonstration Requirements

- Authentication required before device use
- Unauthorized attempts logged
- Secure read/write validation

Challenge Goal (Advanced)

Add token-based authentication with hash validation.

Project 13: USB Network Adapter Monitoring Driver

Goal

Create a monitoring driver for USB Ethernet or WiFi adapters.

Industry Relevance

- Network infrastructure monitoring
- Embedded networking systems
- Industrial IoT

Technical Focus

- Network device monitoring
- Kernel statistics reporting
- Character device communication

Technical Challenges

- Reading network statistics
- Detecting interface state changes
- Synchronizing with user-space monitoring tool

Resources Needed

- USB WiFi or Ethernet adapter
- Raspberry Pi 4

Demonstration Requirements

- Driver displays adapter connection state
- Packet count statistics shown
- Logging through *dmesg*

Challenge Goal (Advanced)

Implement intrusion detection threshold alerts.

Project 14: Multi-Device System Health Driver

Goal

Develop a unified character device driver monitoring CPU load, memory usage, and connected USB devices.

Industry Relevance

- System monitoring appliances
- Industrial edge devices
- Infrastructure health management

Technical Focus

- Kernel system statistics
- Multi-source device monitoring
- User-space dashboard application

Technical Challenges

- Aggregating multiple metrics
- Efficient kernel memory usage
- Polling and synchronization

Resources Needed

- Raspberry Pi 4
- Multiple USB peripherals

Demonstration Requirements

- User application displays live health metrics
- Kernel logs critical alerts
- Monitoring accessible via `/dev/syshealth`

Challenge Goal (Advanced)

Add secure REST API reporting service.

Project 15: USB File Transfer Activity Driver

Goal

Implement a driver that monitors file transfer activity to removable USB storage devices.

Industry Relevance

- Data loss prevention
- Cybersecurity auditing
- Secure storage systems

Technical Focus

- Block device monitoring
- File activity tracking
- Logging and auditing

Technical Challenges

- Detecting file write operations
- Tracking transfer sizes
- Maintaining low overhead

Resources Needed

- USB thumb drive
- Raspberry Pi 4

Demonstration Requirements

- Driver displays file transfer statistics
- User-space application shows transfer logs
- Kernel event logging using *printk()*

Challenge Goal (Advanced)

Detect suspicious mass-copy behavior and trigger alerts.

Project 16: Sense HAT LED Matrix Notification Driver

Goal

Develop a character device driver to control the Sense HAT LED matrix from user-space applications.

Industry Relevance

- Industrial status indicators
- Embedded display systems
- IoT alert systems

Technical Focus

- Character device commands
- LED matrix rendering
- *ioctl()* control operations

Technical Challenges

- Matrix refresh timing
- Multi-pattern display control
- Kernel-user synchronization

Resources Needed

- Sense HAT
- Raspberry Pi 4

Demonstration Requirements

- Display text or symbols through */dev/senseled*
- User application updates LED patterns
- Logging of display operations

Challenge Goal (Advanced)

Implement remote notification control via MQTT.

Project 17: USB Device Usage Accounting Driver

Goal

Create a character driver that records usage duration and access frequency of USB devices.

Industry Relevance

- Asset management
- IT auditing
- Device lifecycle analytics

Technical Focus

- USB event monitoring
- Kernel timing functions
- Character device reporting

Technical Challenges

- Time tracking accuracy
- Device identification persistence
- Handling repeated insertions/removals

Resources Needed

- USB devices
- Raspberry Pi 4

Demonstration Requirements

- Driver records usage duration
- User application generates simple usage reports
- Kernel logs display statistics

Challenge Goal (Advanced)

Export reports securely to remote management server.